The Occam's LGS full chain

LangSplat SAM ViT-H features, Occam's lift, our query — run, measurements, and comparison with our previous results

Radiance FYP — semantic embedding
intellisense08 (RTX 2080, 8 GB). Run 13–14 October 2026; report 16 October 2026

Abstract

This document describes, step by step, the run of Occam's LGS as its authors use it, on our two scenes. The 2D language features come from LangSplat's preprocessing: SAM ViT-H masks at three granularities and one OpenCLIP ViT-B/16 embedding per mask. These are lifted onto our trained Gaussians with Occam's centre-pixel estimator, one table column group per granularity, and queried by our unchanged 3D query and evaluator. We compare it with our previous results: our default lift (A teachers, multi-scale CLIP crops), the scale-averaged lift (Old), our SAM variant B2, and the 13 October run of *the same Occam's lift on our A teachers*. That last comparison isolates the 2D features. The full chain finds **25 of 29** objects with **1 of 6** absent objects firing, against 22/29 with 2–3/6 for every previous configuration. Holding the lift fixed, LangSplat's features account for +3 objects and −2 false positives. The cost is one-off: 5.4 h (backroom) and 4.7 h (flightroom) of SAM extraction on the RTX 2080, then 30 s to lift.

Contents

1 Summary **2**

2 The chain, step by step **2**

3 Running it on the RTX 2080 **4**
3.1 Problems met and how they were solved 4
3.2 Time and memory 4

4 What the features look like **5**

5 Comparison with our previous results **5**
5.1 Headline 5
5.2 Per query 6
5.3 Absent objects 6
5.4 Which level the query chooses 6
5.5 Relevancy-floor sweep 8

6 Assessment **8**

7 Reproduction and files **9**

1 Summary

Configuration (Phase 5 sets, floor 0.55)	hits /29	absent firing /6	best hits at 1 FP	AUROC br / fr	median err (m)
Ours, Old teachers	20	2	19	0.85 / 0.90	0.19 / 0.22
Ours, A teachers (default since 8 Oct)	22	2	19	0.87 / 0.90	0.21 / 0.25
Occam's lift on A teachers (13 Oct run)	22	3	—	0.87 / 0.92	0.26 / 0.18
Occam's full chain (this run)	**25**	**1**	**25**	**0.92 / 1.00**	0.27 / 0.30

1. **Best result measured on our scenes.** Backroom 12/13, flightroom 13/16, development set 5/5.
2. **The features are the cause.** The same Occam's lift on our A teachers gave 22/29 and 3/6 false positives. Replacing only the 2D features with LangSplat's gives 25/29 and 1/6. On 13 Oct, replacing only the lift changed nothing.
3. **Gains:** *swivel chair*, *camera tripod*, *hose reel* (backroom) relative to A, with no losses. *Office chair*, which Occam's lift on A missed, is found again. Flightroom's absent *recycling bin* no longer fires.
4. **Robust to the floor:** 25/29 with one false positive at every floor from 0.52 to 0.55; 24/29 at 0.575.
5. **Costs:** localisation error +0.05 m (median), and about 5 GPU-hours of feature extraction per scene.
6. **Still missed by everything:** white bucket, keyboard, floor lamp, drone gate.

2 The chain, step by step

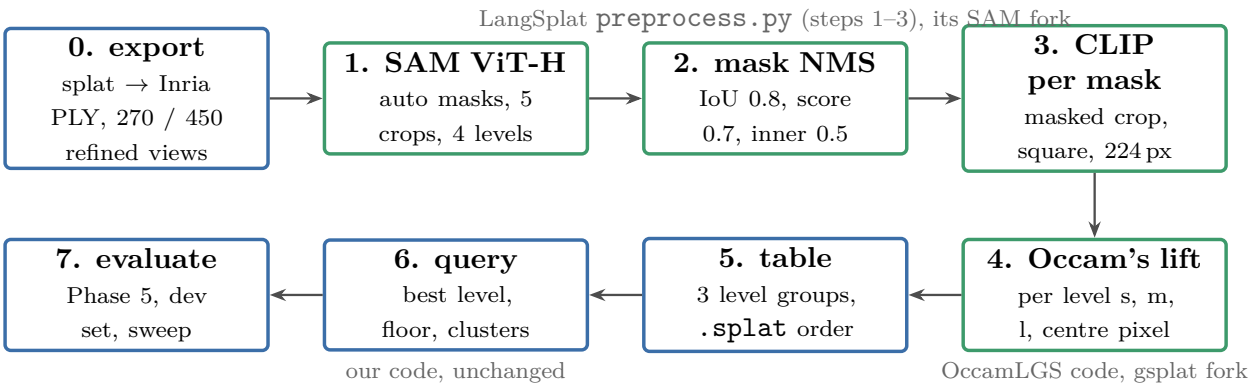


Figure 1: The full chain. Green: the authors' code (LangSplat preprocessing, Occam's lift). Blue: our export, table format, query and evaluator, unchanged from the earlier experiments.

Step 0 — Export (`export_scene.py`, kitchen env)

The trained splatfacto Gaussians (backroom 532,361, flightroom 379,684) are written in the Inria PLY layout in checkpoint order. The training views are written with the poses splatfacto refined (SO3×R3 camera optimiser), the same 270 and 450 views our own lift uses. The images are linked unchanged.

Steps 1–3 — LangSplat's language features (`langsplat_features.py`, `occamlgs` env)

Occam's paper lifts the feature maps LangSplat's `preprocess.py` produces. The runner imports that script's functions and calls them with LangSplat's settings:

Image size	height $>$ 1080 is rescaled to 1080 (LangSplat's rule): backroom 1920×1080, flightroom 1280×720
SAM	ViT-H (`sam_vit_h_4b8939`), LangSplat's fork of `segment-anything`
Automatic generator	32 points per side, crop layers 1 (full image + 4 crops), predicted-IoU 0.7, box-NMS 0.7, stability 0.85, min region 100 px
Granularities	the fork returns SAM's masks as *default*, s (subpart), m (part), l (whole)
Mask NMS	per level: score = stability × predicted IoU; IoU $>$ 0.8, score $<$ 0.7, inner-overlap 0.5 removed
CLIP input	mask's bounding box, pixels outside the mask set to black, padded square, resized to 224 px
CLIP	OpenCLIP ViT-B/16 `laion2b_s34b_b88k` (the same model as our teachers), unit-norm, fp16
Output per image	`<stem>_s.npy`: seg maps [4, H, W] (mask ids, -1 = none); `<stem>_f.npy`: features [masks, 512]



Figure 2: LangSplat's three granularities on one training image per scene (left: backroom frame 141; right: flightroom frame 249). In each panel: top-left the image, top-right level s (subparts), bottom-left m (parts), bottom-right l (whole objects). Each colour is one mask, i.e. one CLIP embedding. Uncovered pixels are dark. The chairs, the bin and the benches are single segments at level l. These whole-object embeddings are what recover the occluded chairs.

Step 4 — Occam's lift (`occam_lift.py -langsplat`, `occamlgs` env)

For each level $\ell \in \{s, m, l\}$ separately, Occam's renderer (gsplat fork) renders every training view at the seg-map resolution. For every Gaussian i blended at the pixel under its projected centre $\lfloor x_i'^v \rfloor$, it records that blend weight w_i^v. Occam's `accumulate_gaussian_feature_per_view` then adds the feature of the mask covering that pixel:

$$f_i^{(\ell)} = \frac{\sum_v w_i^v \, m_\ell^v \, F_\ell^v(\lfloor x_i'^v \rfloor)}{\sum_v w_i^v \, m_\ell^v}, \qquad m_\ell^v = \mathbb{1}[\text{a level-}\ell \text{ mask covers } \lfloor x_i'^v \rfloor] \, .$$

Pixels that no mask covers do not contribute; this is Occam's `get_language_feature` mask. Gaussians never blended at their own centre pixel get no feature. Occam's code prunes them; we keep them as zero rows with weight 0, so table rows stay aligned with the `.splat` file. Seen: backroom 93.7 %, flightroom 91.8 % (our all-pixel lift: 99.9 / 99.6 %).

Step 5 — Table (our `export` step, kitchen env)

The three levels are stored as the table's three scale groups (`clip_scales`), rows unit-normalised, in `.splat` order (`lift_occamfull`, 2.0 GB backroom, 1.5 GB flightroom). The mean of the three is stored as the flat `clip` row.

Step 6 — Query (our `run_query`, unchanged)

For a phrase, LERF relevancy (canonical phrases object / things / stuff / texture, temperature 10) is computed for each level. The level whose top-100 relevancy is highest is used. That is LangSplat's own evaluation rule ("choose the level with the highest relevancy"), and the same rule our A teachers use for crop scales. Then: floor 0.55, relative threshold $\tau = 0.55 + 0.5(\text{peak} - 0.55)$, 0.1 m voxel clusters of at least 15 Gaussians, top cluster, approach point at 1 m.

Step 7 — Evaluation

The Phase 5 sets frozen on 6 Oct (backroom amended 7 Oct): 13 + 16 positives, 3 + 3 absent objects. A query hits if the top cluster lies within 0.75 m of an annotated instance or inside its padded box. The relevancy-floor sweep (0.46–0.60) and the 5-query development set complete it. Nothing was tuned on these sets.

3 Running it on the RTX 2080

3.1 Problems met and how they were solved

LangSplat and Occam's LGS were developed on 48 GB GPUs. Three issues came up on the 8 GB card. Each fix changes only memory use or speed, not the numbers:

1. **SAM decoder out of memory** on the second image: 64 prompt points per batch at 1080p. SAM decodes every point independently and merges batches before filtering, so `points_per_batch` 16 gives the same masks.
2. **Mask NMS out of memory, then infeasible.** After 18 backroom images, one image produced 647 masks at one level. LangSplat's NMS compares every pair of full-resolution masks in a Python loop (5 GB as a GPU copy; days per image on the CPU at that count). It is replaced by a vectorised version: a pairwise-intersection matrix of integer pixel counts, summed over pixel chunks on the GPU, with LangSplat's IoU and inner-overlap rules applied elementwise in the same order. Checked against LangSplat's own function on real masks (table below): identical selection on all 10 level sets, 10–40× faster.
3. **Feature maps in memory.** A per-pixel 512-d map at 1080p is 4.2 GB in fp32. It is held in fp16, the precision LangSplat stores the features in.

The run then resumed from image 19, skipping the finished files. LangSplat's `create()` hides the real exception behind `ValueError(1)`; that message in the first log is the NMS out-of-memory error.

Backroom training view, level	masks	kept (ours)	kept (LangSplat)	time ours / LangSplat (s)
view 1, default / s / m / l	180 / 143 / 89 / 27	58 / 109 / 46 / 22	identical	0.5 / 7.6
view 2, default / s / m / l	305 / 240 / 160 / 60	101 / 168 / 98 / 39	identical	0.9 / 21.9
view 19, m / l	336 / 142	209 / 101	identical	0.4 / 16.2
view 19, default / s	647 / 496	247 / 319	(loop not run)	1.5

3.2 Time and memory

Table 1: Extraction and lift. Per-image times are wall time of LangSplat's steps (SAM, NMS, CLIP); the sum includes the first 18 backroom images from the first attempt. Peak GPU memory 5.9 GB.

Scene	image size	images	median s/img	p90 s/img	total (h)	lift (s)	seen (%)
backroom	1920×1080	270	70	95	5.4	34	93.7
flightroom	1280×720	450	37	42	4.7	28	91.8

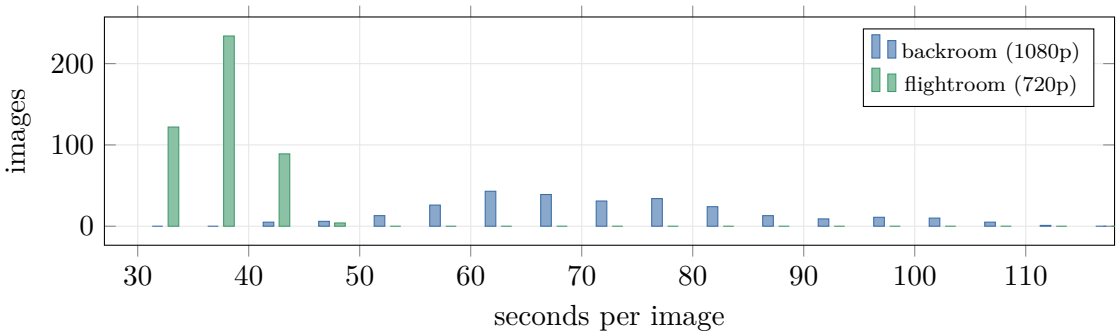


Figure 3: LangSplat extraction time per image. SAM's automatic generator (5 crops × 1,024 prompts) dominates.

4 What the features look like

Table 2: Masks kept per image after NMS, per level (median and 10–90th percentile), and the share of pixels covered by some mask (median, minimum over images). Levels s, m, l are lifted; *default* is computed by LangSplat but not used by Occam's lift.

Scene	level	lifted	masks (median)	masks (p10–p90)	covered (%)	min covered (%)
backroom	default		146	82–252	95.1	78.4
	s	✓	213	130–346	87.7	26.9
	m	✓	128	73–223	94.6	77.4
	l	✓	56	28–102	89.7	41.2
flightroom	default		66	33–108	95.2	78.2
	s	✓	114	69–166	90.2	34.2
	m	✓	57	30–91	93.7	65.2
	l	✓	21	9–34	90.2	34.2

Level s splits objects into parts (median 213 masks per backroom image), and l keeps whole objects (56). Each level leaves 5–12

,
Old teachers instead give every pixel a crop embedding, background included. In the table, 91–94 % of rows have a feature at each level.

5 Comparison with our previous results

5.1 Headline

Table 3: Phase 5 evaluation, query settings unchanged since Phase 1. Per scene: hits, median and 90th-percentile localisation error (m), absent objects returning a candidate, AUROC of peak relevancy (positives vs absent).

			backroom (13 + 3)					flightroom (16 + 3)				
Configuration	hits	FP	hits	err	p90	FP	AUROC	hits	err	p90	FP	AUROC
Occam full	**25**/29	1/6	12/13	0.27	0.62	1/3	0.92	13/16	0.30	0.80	0/3	1.00
Occam lift, A	22/29	3/6	9/13	0.26	1.59	2/3	0.87	13/16	0.18	0.65	1/3	0.92
Ours A	22/29	2/6	9/13	0.21	1.73	1/3	0.87	13/16	0.25	0.72	1/3	0.90
Ours Old	20/29	2/6	10/13	0.19	0.48	1/3	0.85	10/16	0.22	0.75	1/3	0.90
B2	21/29	2/6	9/13	0.19	3.27	1/3	0.87	12/16	0.20	0.73	1/3	0.94

Same lift, different features. "Occam lift, A" is the 13 Oct run: Occam's own lift code on our A teachers (multi-scale CLIP crops, three scale groups). "Occam full" is the same lift on LangSplat's features (three SAM levels). The only difference between the two rows is the 2D features, and it is worth +3 hits (22 → 25), −2 absent objects firing (3 → 1), and higher AUROC on both scenes (0.87 → 0.92, 0.92 → 1.00).

5.2 Per query

- **Occluded and thin objects gain most.** Flightroom's armchair (0.58 → 0.71 against A) and upholstered chair (0.55 → 0.65) sit behind the whiteboard in most views. A crop of fixed size around them contains mostly whiteboard; a SAM mask contains only the chair. The camera tripod (thin legs) rises from 0.54 to 0.59, just enough to clear the floor and form a cluster.
- **Swivel chair and office chair are both found** at level l. Our A table found the office chair but not the swivel chair; Occam's lift on A found neither.
- **Hose reel**, mounted on the ceiling and lost when A replaced Old, is found again at level l, and the camera tripod at level m.
- **Remaining misses.** White bucket: wrong object 1.8 m away. Keyboard: peak 0.556, just over the floor but no cluster of 15 Gaussians. Floor lamp and drone gate: the right area, but the cluster centroid 0.87 and 0.97 m off. No configuration in either experiment has found these four.
- **Some peaks fall** (flightroom red cup 0.75 → 0.67, poster 0.74 → 0.67) without losing the hit. A mask-level embedding of a small object is less distinctive than a tight crop at the right scale, but still enough.

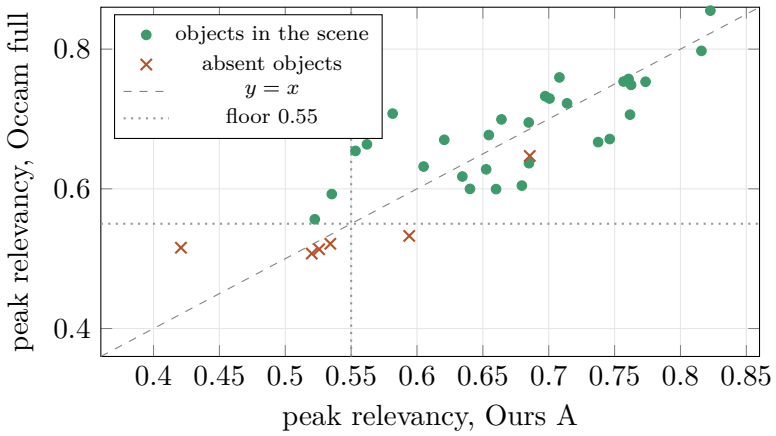


Figure 4: Peak relevancy per query, our A lift against the full chain (both scenes). Points above the diagonal rose. The absent objects (crosses) moved down or stayed; flightroom's recycling bin crossed below the floor.

5.3 Absent objects

Only backroom's *ceiling fan* still fires (peak 0.65). It fires in every configuration and at every floor, which makes a real look-alike in the room likely (HANDOVER §2.5); it should be verified in the photos.

5.4 Which level the query chooses

The query never picks the subpart level for an object in the scene, as A never picks its small-crop group. Whole objects (l) are chosen for 22 of 29 queries. Level l also separates best: on flightroom its

Table 4: Per positive query: hit (•) or miss (○) and peak relevancy for four configurations; for the full chain also the chosen SAM level, the error to the nearest instance (m) and the failure kind. Bold queries differ between configurations.

	Occam full	Occam lift, A	Ours A	Ours Old	level	err	failure (full)
backroom							
recycling bin	• 0.75	• 0.79	• 0.77	• 0.72	m	0.07	
blue trash can	• 0.75	• 0.77	• 0.76	• 0.69	l	0.18	
office chair	• 0.68	○ 0.68	• 0.65	• 0.56	l	0.37	
swivel chair	• 0.67	○ 0.65	○ 0.62	○ 0.54	l	0.36	
grandfather clock	• 0.72	• 0.72	• 0.71	• 0.67	l	0.41	
clock	• 0.73	• 0.70	• 0.70	• 0.66	l	0.22	
children's road play mat	• 0.80	• 0.80	• 0.82	• 0.76	l	0.17	
camera tripod	• 0.59	• 0.55	○ 0.54	○ 0.51	m	0.67	
purple foam mat	• 0.76	• 0.76	• 0.76	• 0.72	m	0.21	
white bucket	○ 0.63	○ 0.62	○ 0.60	○ 0.56	l	1.80	wrong object
glass door cabinet	• 0.76	• 0.73	• 0.71	• 0.65	m	0.27	
hose reel	• 0.64	○ 0.69	○ 0.69	• 0.63	l	0.09	
moving box	• 0.71	• 0.76	• 0.76	• 0.70	l	0.28	
flightroom							
armchair	• 0.71	• 0.59	• 0.58	○ 0.53	l	0.31	
upholstered chair	• 0.65	• 0.56	• 0.55	○ 0.51	l	0.30	
whiteboard	• 0.75	• 0.77	• 0.76	• 0.71	l	0.15	
computer monitor	• 0.70	• 0.67	• 0.66	• 0.62	l	0.12	
keyboard	○ 0.56	○ 0.53	○ 0.52	○ 0.49	l	–	no candidate
floor lamp	○ 0.66	○ 0.58	○ 0.56	○ 0.51	l	0.87	bad position
red cup	• 0.67	• 0.71	• 0.75	• 0.71	m	0.03	
water bottle	• 0.63	• 0.61	• 0.65	• 0.60	l	0.03	
water jug	• 0.62	• 0.62	• 0.63	○ 0.59	l	0.13	
round table	• 0.60	• 0.64	• 0.64	• 0.58	l	0.57	
drone gate	○ 0.60	○ 0.70	○ 0.68	○ 0.61	l	0.97	bad position
PVC pipe frame	• 0.86	• 0.83	• 0.82	• 0.75	l	0.69	
poster	• 0.67	• 0.74	• 0.74	• 0.64	m	0.24	
garden cart	• 0.73	• 0.69	• 0.70	• 0.62	l	0.40	
wheelbarrow	• 0.69	• 0.69	• 0.68	• 0.62	l	0.35	
cardboard box	• 0.60	• 0.66	• 0.66	• 0.62	m	0.17	

Table 5: Absent objects: whether a candidate is returned (fires) and the peak relevancy.

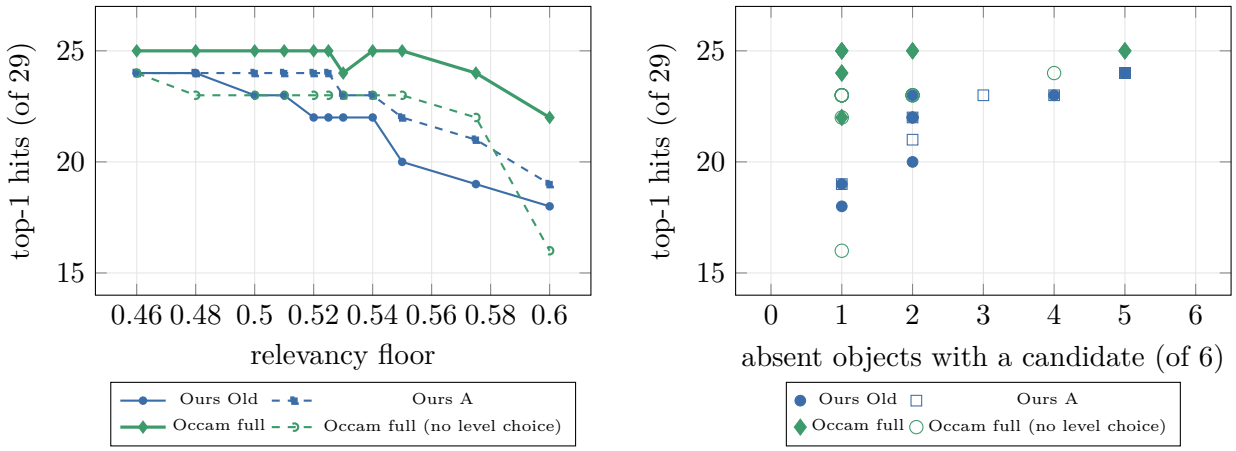
Absent object	Occam full	Occam lift, A	Ours A	Ours Old
bicycle (backroom)	−0.52	fires 0.56	−0.53	−0.50
sofa (backroom)	−0.52	−0.43	−0.42	−0.39
ceiling fan (backroom)	fires 0.65	fires 0.69	fires 0.69	fires 0.66
red tool chest (flightroom)	−0.51	−0.51	−0.52	−0.50
recycling bin (flightroom)	−0.53	fires 0.60	fires 0.59	fires 0.54
bicycle (flightroom)	−0.51	−0.51	−0.53	−0.48

Table 6: Level or scale group chosen per positive query (counts). SAM levels s / m / l for the full chain; crop-scale groups small / medium / large for the A teachers.

	backroom (13)			flightroom (16)		
Configuration	s / S	m / M	l / L	s / S	m / M	l / L
Occam full (SAM levels s / m / l)	0	4	9	0	3	13
Occam lift, A (scale groups S / M / L)	0	5	8	0	6	10
Ours A (scale groups S / M / L)	0	4	9	0	7	9

median positive peak (0.66) is 0.13 above the highest absent peak, against 0.06 at level s. Choosing among the three adds two hits over averaging them (the flat variant: 23/29 at one false positive, §5.5).

5.5 Relevancy-floor sweep



(a) Hits vs floor.

(b) Hits vs false positives (one point per floor).

Figure 5: Floors 0.46–0.60, both scenes combined. Old and A from the 13 Oct sweep.

At one false positive the full chain finds 25/29, against 19/29 for Old and A. At five false positives (floor 0.46) Old and A reach 24/29 and the full chain stays at 25. It stays at 24–25 from floor 0.46 to 0.575, while Old's and A's trade hits for false positives throughout. These floors are read off the evaluation set, which is optimistic for every curve alike.

6 Assessment

What changed and why it helps. The previous designs embed square crops of fixed sizes: a 7-scale pyramid averaged (Old) or kept in three groups (A). Such a crop mixes the object with whatever surrounds it. LangSplat embeds each SAM mask alone, on black, at the scale of the object.

Table 7: Full chain: per level, median peak over the positive queries and highest peak over the absent objects.

	level s		level m		level l	
Scene	pos. median	absent max	pos. median	absent max	pos. median	absent max
backroom	0.675	0.647	0.705	0.645	0.722	0.621
flightroom	0.574	0.509	0.626	0.513	0.658	0.533

The CLIP embedding therefore describes the object rather than a neighbourhood. That matters most when the object is occluded (the chairs behind the whiteboard) or thin (the tripod), exactly where the crop designs failed. It also explains the better rejection of absent objects: a segment-level embedding of a bin-shaped object in flightroom is less "recycling bin"-like than a crop containing bin-like context.

Caveats. The improvement is three queries on 29 (3 gains, 0 losses against A; exact sign test $p = 0.25$). It is supported by fewer false positives, higher AUROC and a flat floor curve, but a larger, independently annotated query set is still needed. The annotations were placed by Claude from photos, independently of all methods. Median localisation error is 0.05 m higher, because whole-object segments spread relevancy over the object's far side. *default*-level masks are computed but unused, as in Occam's code.

Relation to our B2. B2 (8 Oct) also used SAM regions: ViT-B, levels by mask area, pyramid features where no mask covers a pixel. It reached 21/29. Four differences remain untested one by one: ViT-H with a crop layer, SAM's own granularities, skipping uncovered pixels instead of filling them, and LangSplat's mask NMS. With the LangSplat masks now cached, each can be checked by re-lifting (30 s) instead of re-extracting (hours).

Recommendation. Make region-level features the base of `semantic/new`: SAM masks at three granularities, one CLIP embedding per masked crop, a training-free lift per level, and the level chosen per query. Next steps, in order:

1. Ablate skip vs fill on the cached masks.
2. Reduce extraction cost: no crop layer, fewer prompts, SAM 2 or an efficient SAM, on the RTX 5060 Ti.
3. Anchor the goal on the object's visible surface to recover the 0.05 m.
4. Address the four objects no design finds, with a stronger recogniser or a re-ranker on the top candidates.

7 Reproduction and files

```
# branch semantic/LGS, repo root
source semantics/baselines/env.sh occamlgs
python semantics/baselines/langsplat_features.py --scene backroom          # 5.4 h on the RTX 2080 (resumable)
python semantics/baselines/occam_lift.py --scene backroom \
       --langsplat ~/Radiance/third_party/langsplat_features --suffix _occamfull
source ~/Radiance/figs/figs_env.sh
python figs/semantic_pipeline.py --scene backroom --backend lift --table-suffix _occamfull \
       --clip-mode scales --only export
cd semantics && python -m radiance_semantics.evaluate --scene backroom --variants L-C@occamfull
```

- Raw results: `../../raw/eval/` (Phase 5), `raw/eval_dev/`, `raw/occam/` (levels chosen, masks per image, timing, table coverage), `raw/logs/` (run logs and the NMS check).

- Scripts: `../../scripts/occam_analysis.py` (kitchen, CPU) and `make_occam_tables.py` regenerate every table and plot here.
- Host: the LangSplat cache (15 GB) at `~/Radiance/third_party/langsplat_features/`; tables `lift_occamfull` under each scene's `semantics/<run>/`.
- The joint report on both papers' full pipelines (with LUDVIG): `../full_pipelines_report.pdf`.